Easy Keybinding User Guide

Please read carefully before using this asset!

Introduction

The Easy Keybinding asset is designed to make adding keybinding features to your project easy. This asset will allow you to add a keybinding panel to your game options so that the user can configure the keyboard and mouse keys to be used for different game actions. There is some additional coding required by the developer, but it is all explained in this guide and in the comments in the code to make it easy to implement. While there are many steps to take in order to make this work, it is still much easier than creating all of this from scratch, which can take many days and lots of frustration. Follow the instructions carefully. Do not skip any steps.

To give you a better idea of what this project can do, open the KeyBinding scene in Unity and press play. Press the various keys to see the indicators turn green. Press the Controls button to open the KeyBinding panel. This panel contains buttons that let you configure each key. Try changing all the keys (using keyboard keys or mouse buttons) and pressing save. Then press the new keys and see the indicators turn green. Open the KeyBinding panel again and click on Reset and Save. This will restore the default controls.

Adding Keybinding to Your Project

1. Download the Easy Keybinding asset from the Asset Store and import it into your project.

2. In your project, open the scene that will have the key binding feature. Create an empty game object and name it InputController. In the inspector, attach the script InputController to the new InputController object. The InputController script is located in the Scripts folder of the Easy Keybinding project. In addition, if your project does not have an Event System, create one by right clicking on the hierarchy and selecting UI > Event System. This will enable button clicking.

3. Open the InputController script. Steps 4-11 are all performed inside the InputController script. Do not be scared by editing this script, it's really easy, it's mostly just copying, pasting and modifying what is already there, or deleting unnecessary lines. Unfortunately, these edits are needed because every project has different keys and actions. The steps below will guide you.

4. In the “Key binding indices” comment section near the top, modify the comments so that there is a unique index assigned to each key you want to configure. The ones shown are only for the sample scene, you can modify them, delete them or add more depending on your project’s needs. Use this comment section to keep track of each index.

```
// Key binding indices:  Each key
/*
Up - 0
Down - 1
Left - 2
Right - 3
Jump - 4
Fire 1 - 5
Fire 2 - 6
Fire 3 - 7
*/
```

5. Under the header “Indicators (for sample scene only)”, delete all the Image objects and the header line. These are only for the sample scene and are not needed for your project. These items will eventually be replaced by whatever objects you want your keys to activate in your project. You will get some compiler errors after this. Ignore them because they will go away after you complete step 9.

```
// These are the indicators used for the sample Ke
[Header("Indicators (for sample scene only)")]
[SerializeField] public Image UpIndicator;
[SerializeField] public Image DownIndicator;
[SerializeField] public Image LeftIndicator;
[SerializeField] public Image RightIndicator;
[SerializeField] public Image JumpIndicator;
[SerializeField] public Image Fire1Indicator;
[SerializeField] public Image Fire2Indicator;
[SerializeField] public Image Fire3Indicator;
```

6. Under the header “Keybinding Button Prefab Text Objects”, add or delete Text items depending on how many keys your project has to configure. Rename the existing ones as needed based on your project’s needs.

```
// The text object in each key binding prefab should be
[Header("Keybinding Button Prefab Text Objects")]
[SerializeField] public Text UpKeyBindingText;
[SerializeField] public Text DownKeyBindingText;
[SerializeField] public Text LeftKeyBindingText;
[SerializeField] public Text RightKeyBindingText;
[SerializeField] public Text JumpKeyBindingText;
[SerializeField] public Text Fire1KeyBindingText;
[SerializeField] public Text Fire2KeyBindingText;
[SerializeField] public Text Fire3KeyBindingText;
```

7. Under the “Initializes the key codes” comment, initialize a KeyCode for each key you want to configure. Add new key codes as needed or modify the existing ones. The recommended name for the key code is the action that will be taken when pressing that key. Since the key binding can be changed, do not include an actual keyboard letter, only the name of the action.

```
// Initializes the key codes.
KeyCode UpKeyCode;
KeyCode DownKeyCode;
KeyCode LeftKeyCode;
KeyCode RightKeyCode;
KeyCode JumpKeyCode;
KeyCode Fire1KeyCode;
KeyCode Fire2KeyCode;
KeyCode Fire3KeyCode;
```

8. Under the Start() method, edit the KeyBindings.DefaultKeyBindArray with your project’s factory default key bindings. Use the link in the comments to find the correct key names for each key from the Unity documentation. **The position of each key in the array should match the index from step 4.** This array will be used to assign key bindings when the user opens the program for the first time or when the Reset button is pressed. It’s also used if the file that saves the key bindings is not found.

```
void Start()
{
    // Set the factory default key bindings here.  This array will set the key bindings for new users who opened
    // Every key you plan to use must be assigned a factory default.
    // See the KeyCode page in the Unity documentation for the various key names that can be used here:  https://
    KeyBindings.DefaultKeyBindArray = new string[] { "W", "S", "A", "D", "Space", "Mouse0", "Mouse1", "Mouse2" };

    // Loads the saved key bindings
    LoadKeyBindings();
}
```

9. Under the Update() method, delete all the actions that are for the sample scene. Replace these actions with the ones for your project. Remember to include both GetKeyDown and GetKeyUp for each key. GetKeyDown is called when the key is pressed, and GetKeyUp is called when the key is released. The argument for these should be the key codes you defined in the previous step.

```
// Update is called once per frame
void Update()
{
    // This section decides what actions are taken when

    // Executes when the up key is pressed
    if (Input.GetKeyDown(UpKeyCode))
    {
        UpIndicator.color = Color.green;
    }

    // Executes when the up key is released
    if (Input.GetKeyUp(UpKeyCode))
    {
        UpIndicator.color = Color.white;
    }
```

10. Under the AssignKeyBindings() method, modify the lines based on your project's needs. There should be a line for each key code that is defined. Copy, paste and modify these lines as needed. At the end of the line, each line should have a unique index for the KeyBindArray[]. **It should be the same index that was used in step 4.**

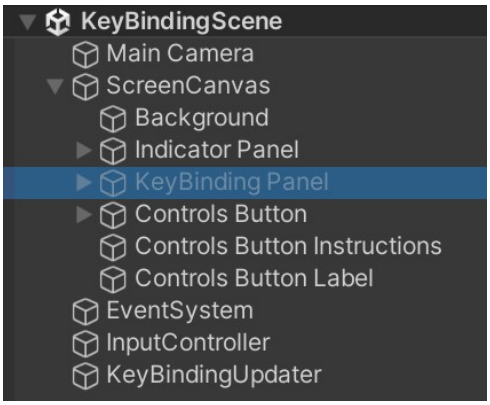
```
// Assigns the key bindings to key codes so they can be called.  Any new key codes should be
void AssignKeyBindings()
{
    UpKeyCode = (KeyCode)System.Enum.Parse(typeof(KeyCode), KeyBindings.KeyBindArray[0]);
    DownKeyCode = (KeyCode)System.Enum.Parse(typeof(KeyCode), KeyBindings.KeyBindArray[1]);
    LeftKeyCode = (KeyCode)System.Enum.Parse(typeof(KeyCode), KeyBindings.KeyBindArray[2]);
    RightKeyCode = (KeyCode)System.Enum.Parse(typeof(KeyCode), KeyBindings.KeyBindArray[3]);
    JumpKeyCode = (KeyCode)System.Enum.Parse(typeof(KeyCode), KeyBindings.KeyBindArray[4]);
    Fire1KeyCode = (KeyCode)System.Enum.Parse(typeof(KeyCode), KeyBindings.KeyBindArray[5]);
    Fire2KeyCode = (KeyCode)System.Enum.Parse(typeof(KeyCode), KeyBindings.KeyBindArray[6]);
    Fire3KeyCode = (KeyCode)System.Enum.Parse(typeof(KeyCode), KeyBindings.KeyBindArray[7]);
}
```

11. Under the OpenKeybindingPanel() method, add or delete the text lines based on your project's needs. Each KeyBinding prefab button text that is instantiated in your KeyBinding panel should be included here. This method will load the correct key binding text for each button when you open the KeyBinding panel. Make sure the correct index is associated with each line. **It should be the same index that was used in step 4.**

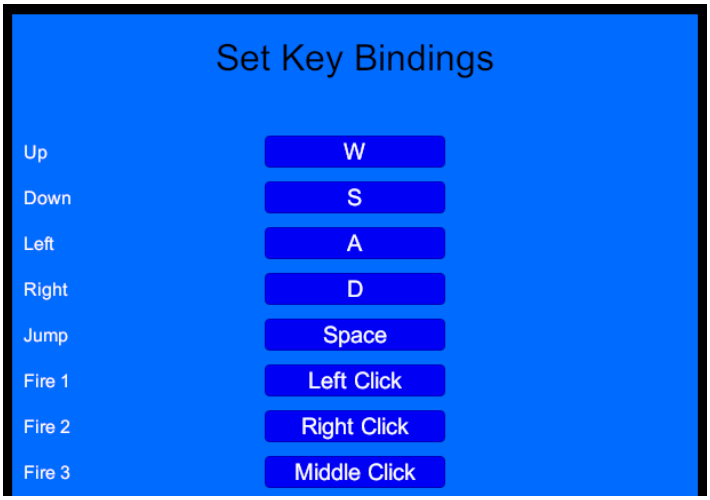
```
// Loads the fields for the key bindings panel when it i
public void OpenKeybindingPanel()
{
    // Do not modify this line
    Array.Copy(KeyBindings.LoadedKeyBindArray, KeyBindin

    // Add, modify or delete these lines based on your p
    UpKeyBindingText.text = KeyBindingText(0);
    DownKeyBindingText.text = KeyBindingText(1);
    LeftKeyBindingText.text = KeyBindingText(2);
    RightKeyBindingText.text = KeyBindingText(3);
    JumpKeyBindingText.text = KeyBindingText(4);
    Fire1KeyBindingText.text = KeyBindingText(5);
    Fire2KeyBindingText.text = KeyBindingText(6);
    Fire3KeyBindingText.text = KeyBindingText(7);
}
```

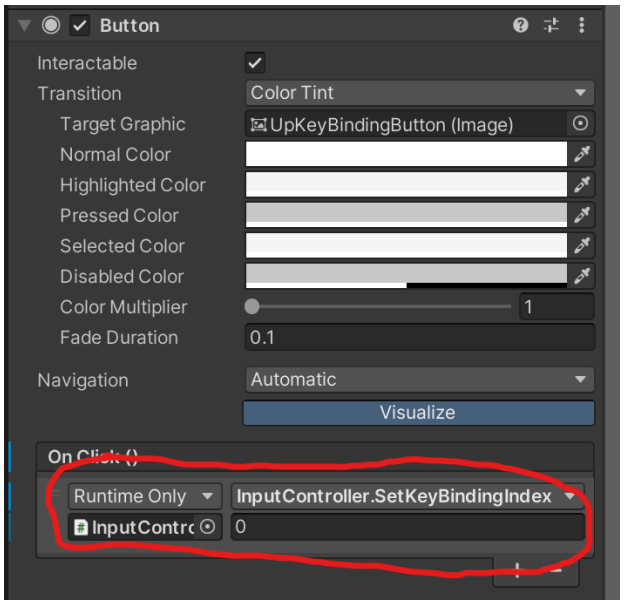
12. Open the scene KeyBindingScene that came with this asset. The scene is located in the Scenes folder of this asset. Click on the triangle to the left of the ScreenCanvas object and then right click on the KeyBinding Panel. Select Copy. Close the scene. Open your project's scene. Paste the KeyBinding panel in your project hierarchy, wherever you think is best. This is the panel that the user will open to configure the key bindings. Modify this panel to match your project's graphics and text.



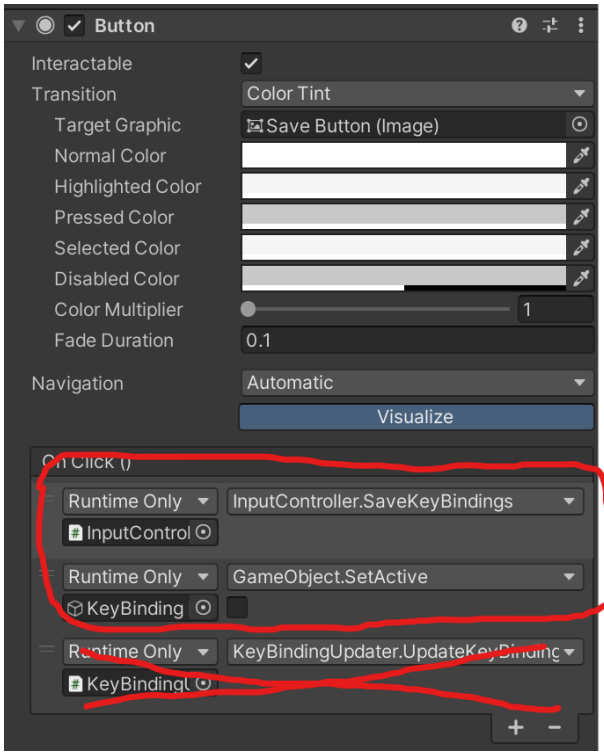
13. The Keybinding Panel comes with 8 sample KeyBinding Button prefabs. These are the prefabs the user will click on to change a key binding. Add or delete prefabs according to your project's needs. The prefab is located in the asset's Prefabs folder, but you can also copy and paste the ones that were copied with the KeyBinding panel. Modify the labels to the left of each prefab based on the action each key will activate. Modify the prefab graphics and text to match your project.



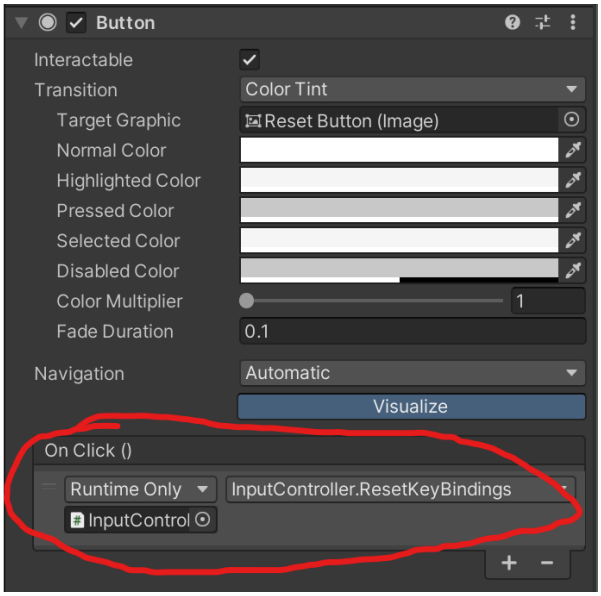
14. Each KeyBinding Button prefab you added requires a unique index to be assigned. Click on a prefab and look at the inspector. Under the Button component, add an action under On Click () (it should already be there in the sample prefabs, so this is only for new prefabs that are instantiated). Drag the InputController game object into this action. In the dropdown, select the SetKeyBindingIndex method and type in the index for this action. **This index should match the index you set in step 4.**



15. Click on the Save Button under the KeyBinding Panel game object. Look at the inspector and scroll down to the Button component. Add 2 actions under On Click (). Drag the InputController object into the first action and select the SaveKeyBindings method. Drag the KeyBindings Panel into the second action and select "GameObject.SetActive" and uncheck the box so it deactivate the object. This button will save the keybindings and close the KeyBinding panel. If you copied the button from the sample project, there may be a third action already on the button. Delete the action that says KeyBindingUpdater.UpdateKeyBinding. It's only for the sample scene.

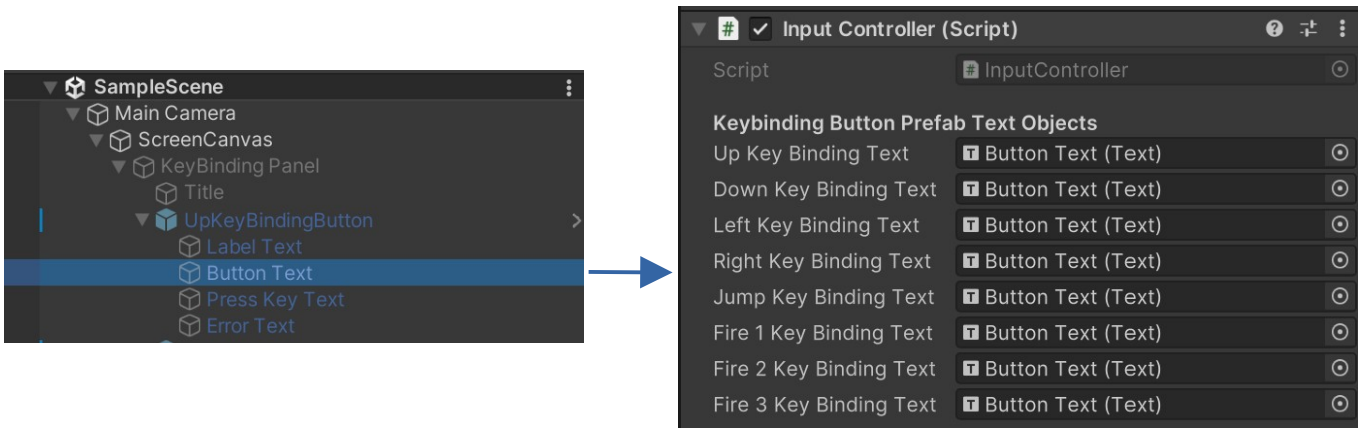


15. Click on the Reset Button under the KeyBinding Panel game object. Look at the inspector and scroll down to the Button component. Add an action under On Click (). Drag the InputController into the action and select the ResetKeyBindings method. This button will reset the key bindings to factory default.

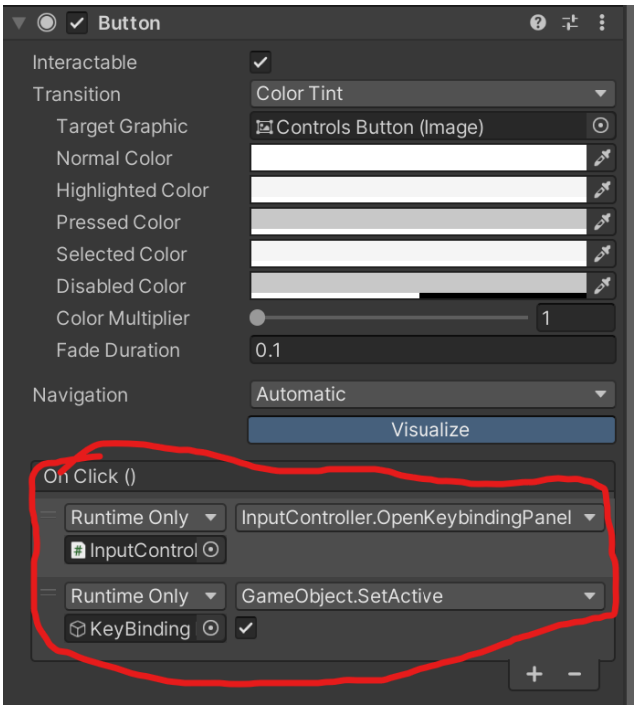


16. Click on the KeyBinding Panel game object and set it as inactive in the inspector. This panel will be off by default and will be turned on when pressing the Controls button which is created in the next step.

17. Click on the InputController game object and look at the inspector. You will see several unassigned objects under the header “Keybinding Button Prefab Text Objects”. Drag the “Button Text” text object here from each Keybinding Button prefab that is instantiated in your KeyBinding panel. This will allow the InputController to update these fields every time you change a key binding or open they KeyBinding panel.



18. Add a “Controls” button to your project for opening the KeyBinding panel. This button is typically located in a game’s Options panel, but you can put it anywhere. Under the Button component in the inspector, add two actions under On Click (). Drag the InputController object into the first action and select OpenKeybindingPanel(). Drag the KeyBinding Panel game object into the second action and Select “GameObject.SetActive” and check the box so it is activated when the button is pressed.



19. The key bindings should be ready to work now. If there are any issues, refer to the KeyBinding scene as a working example that can help you debug any problems.